



**OMCS6250**

# **Computer Networks**

## **Internet Architecture and Design Principles**



With Instructor Dr. Maria Konte

**Internet Protocol Architecture,  
End-2-End Principle, Interconnecting Hosts**



# Learning Objectives

- **Architectural design choices:** The Internet was designed for scalability and evolvability through a **layered protocol stack** (services offered by each layer + encapsulation across layers) and the **end-to-end principle** (a simple core with most functionality at the edges, with practical exceptions like NAT/middleboxes).
- **Describe the Internet's hourglass architecture** and summarize a proposed **evolutionary model** that explains why core protocols become entrenched and hard to replace (ossification).
- **Identify key network devices** and behaviors such as hubs/repeaters, switches/bridges, routers, and explain **learning bridges and spanning tree** for preventing Layer-2 loops.





# The Internet's Growth Drivers & Scaling Challenges

## Infrastructure Expansion, Video Traffic, and AI Applications

- **Users connecting:** In 2025, about 6 billion people (roughly three-quarters of the world) used the Internet, up from about 5.8B in 2024.
- **Devices connecting:** The number of connected IoT devices is estimated to reach about 21.1 billion by end-2025 (around 14% year-over-year growth).
- **Applications exploding:** Increasing demand for app-driven network performance keeps rising into 2026.
- **Content Distribution Networks (CDNs) scaling:** video traffic keeps increasing overall. Ericsson projects video will account for about 76% of all mobile data traffic by the end of 2025. CDN capacity and placement matter more every year.
- **AI and Networking:** AI is improving how networks are managed and optimized, while AI applications are simultaneously driving new demands on network capacity, latency, and data-center connectivity.

# Internet Architecture

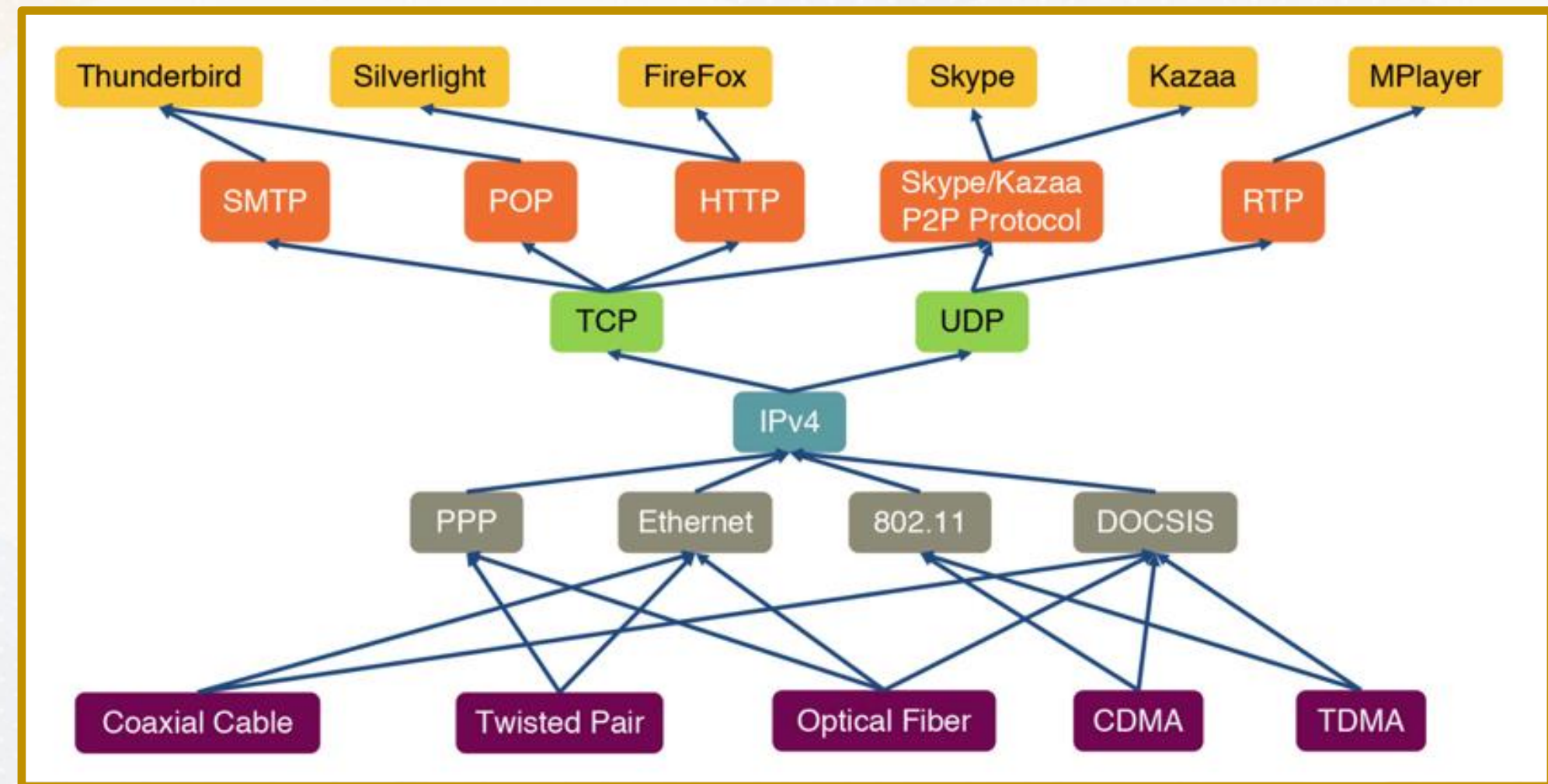
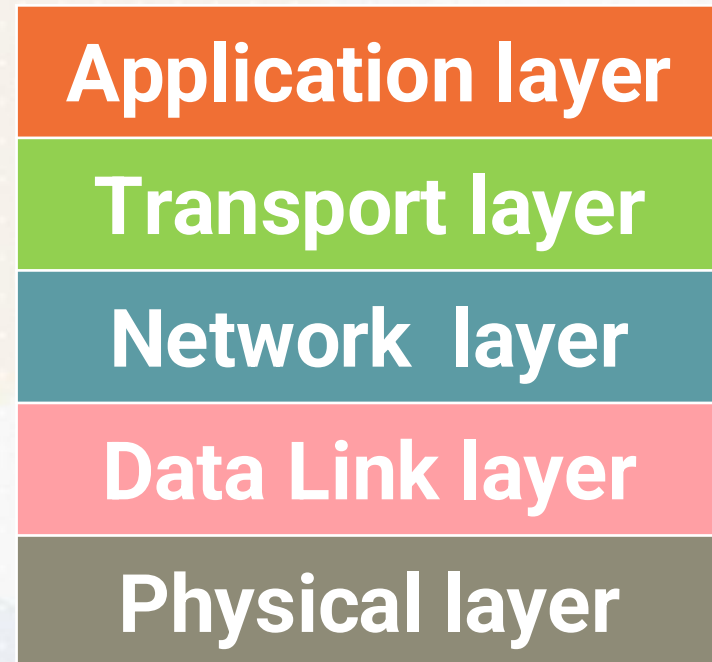
## Why We Need a Layered Architecture

- The Internet evolved from a small research experiment into a global communication infrastructure that **keeps growing in users, devices, and applications.**
- The core challenge: different applications must communicate across heterogeneous technologies (Ethernet, Wi-Fi, cellular, etc.) while seamlessly working.
- The solution is structure: protocol designers organize functionality into layers, where **each layer uses the service from the layer below and provides a service to the layer above.** Layering provides scalability, modularity, and easier evolution).



# The Internet Protocol Stack

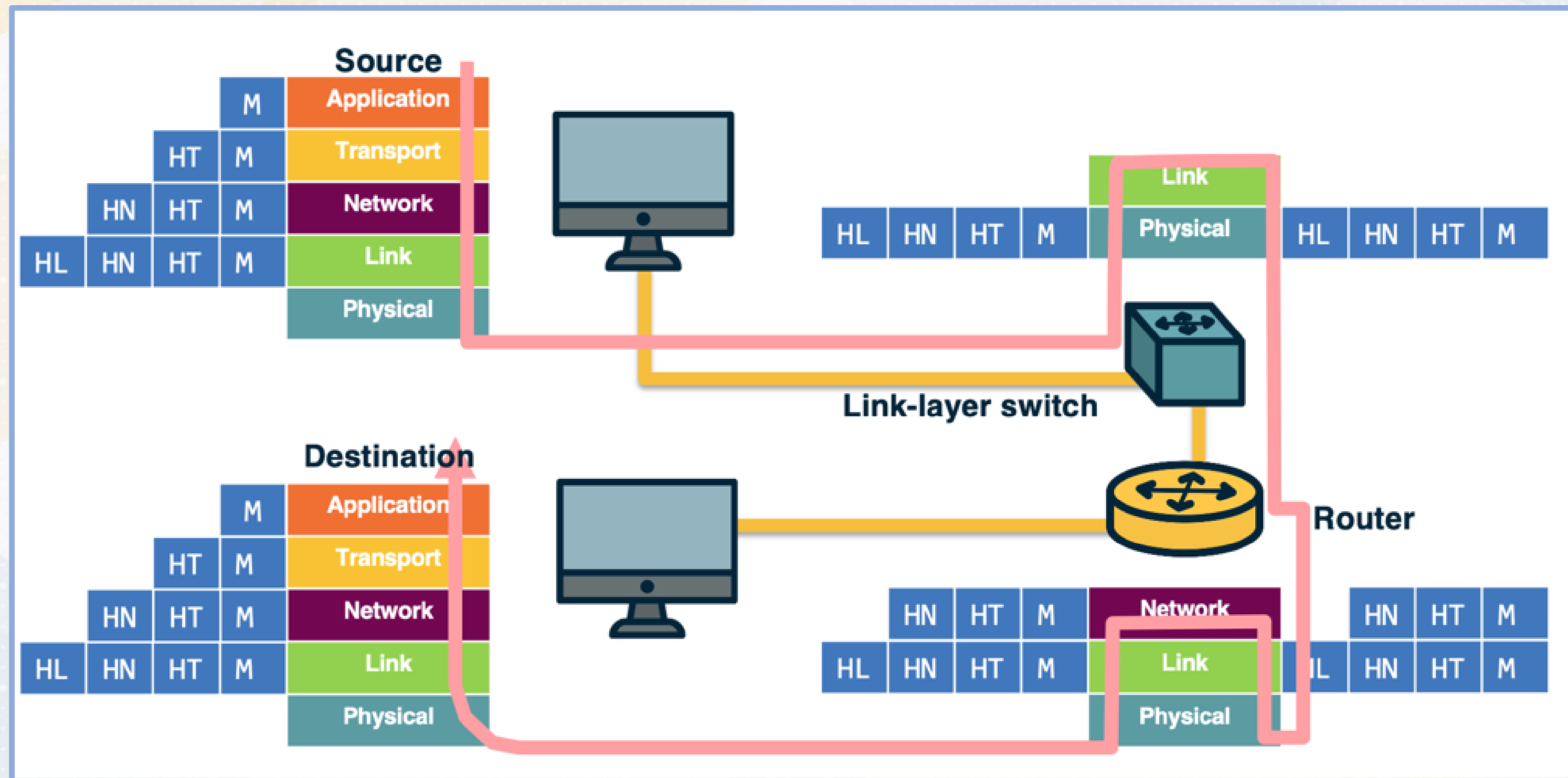
## Five-layered Internet Protocol Stack



Akhshabi, Saamer, and Constantine Dovrolis. "The evolution of layered protocol stacks leads to an hourglass-shaped architecture." *Proceedings of the ACM SIGCOMM 2011 Conference*. 2011.

# How Network Layers Interconnect

## Packet Encapsulation





# The End-to-End Principle (1/2)

“The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the endpoints of the communications system. Therefore, providing that questioned function as a feature of the communications systems itself is not possible.”

## End-To-End Arguments in System Design

J. H. SALTZER, D. P. REED, and D. D. CLARK

Massachusetts Institute of Technology Laboratory for Computer Science

This paper presents a design principle that helps guide placement of functions among the modules of a distributed computer system. The principle, called the end-to-end argument, suggests that functions placed at low levels of a system may be redundant or of little value when compared with the cost of providing them at that low level. Examples discussed in the paper include bit-error recovery, security using encryption, duplicate message suppression, recovery from system crashes, and delivery acknowledgment. Low-level mechanisms to support these functions are justified only as performance enhancements.

CR Categories and Subject Descriptors: C.0 [General] Computer System Organization—*system architectures*; C.2.2 [Computer-Communication Networks]: Network Protocols—*protocol architecture*; C.2.4 [Computer-Communication Networks]: Distributed Systems; D.4.7 [Operating Systems]: Organization and Design—*distributed systems*

General Terms: Design

Additional Key Words and Phrases: Data communication, protocol design, design principles

### 1. INTRODUCTION

Choosing the proper boundaries between functions is perhaps the primary activity of the computer system designer. Design principles that provide guidance in this choice of function placement are among the most important tools of a system



# The End-to-End Principle (2/2)

Keeping the **network core simple** while implement intelligence at the edges (*e.g.* making sure a message arrives correctly or is encrypted) letting innovation flourish and **making systems more flexible and resilient.**

## End-To-End Arguments in System Design

J. H. SALTZER, D. P. REED, and D. D. CLARK

Massachusetts Institute of Technology Laboratory for Computer Science

This paper presents a design principle that helps guide placement of functions among the modules of a distributed computer system. The principle, called the end-to-end argument, suggests that functions placed at low levels of a system may be redundant or of little value when compared with the cost of providing them at that low level. Examples discussed in the paper include bit-error recovery, security using encryption, duplicate message suppression, recovery from system crashes, and delivery acknowledgment. Low-level mechanisms to support these functions are justified only as performance enhancements.

CR Categories and Subject Descriptors: C.0 [General] Computer System Organization—*system architectures*; C.2.2 [Computer-Communication Networks]: Network Protocols—*protocol architecture*; C.2.4 [Computer-Communication Networks]: Distributed Systems; D.4.7 [Operating Systems]: Organization and Design—*distributed systems*

General Terms: Design

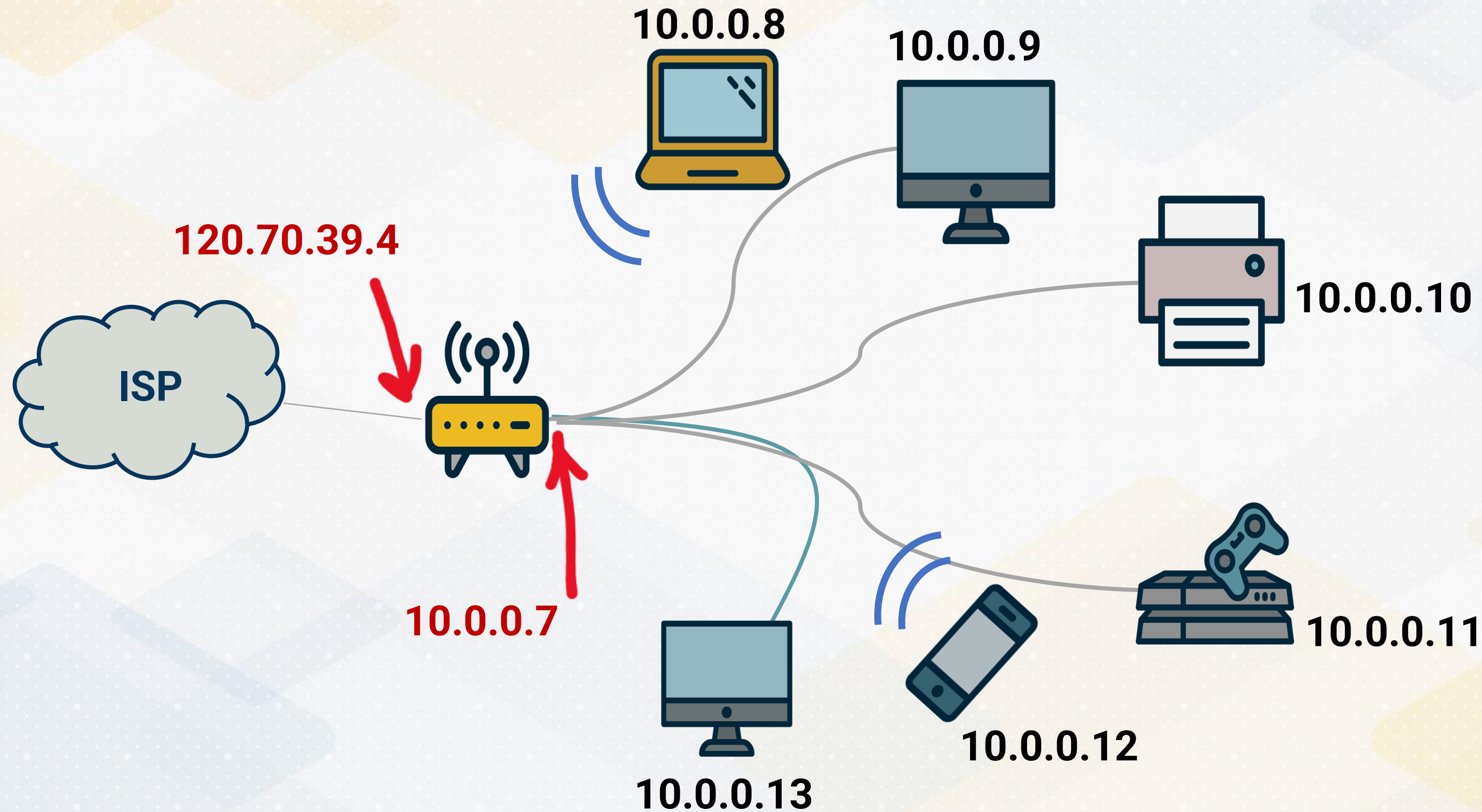
Additional Key Words and Phrases: Data communication, protocol design, design principles

### 1. INTRODUCTION

Choosing the proper boundaries between functions is perhaps the primary activity of the computer system designer. Design principles that provide guidance in this choice of function placement are among the most important tools of a system

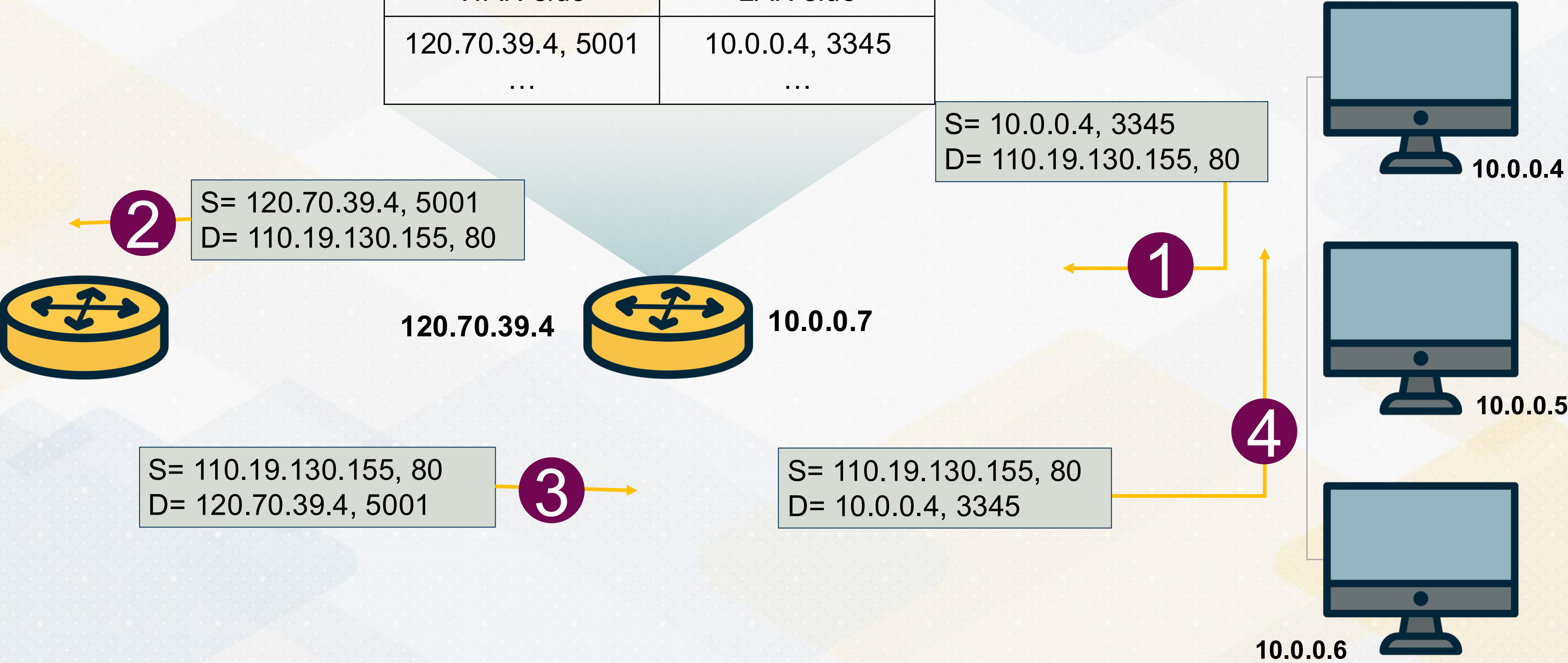


# Violations of The End-to-End Principle



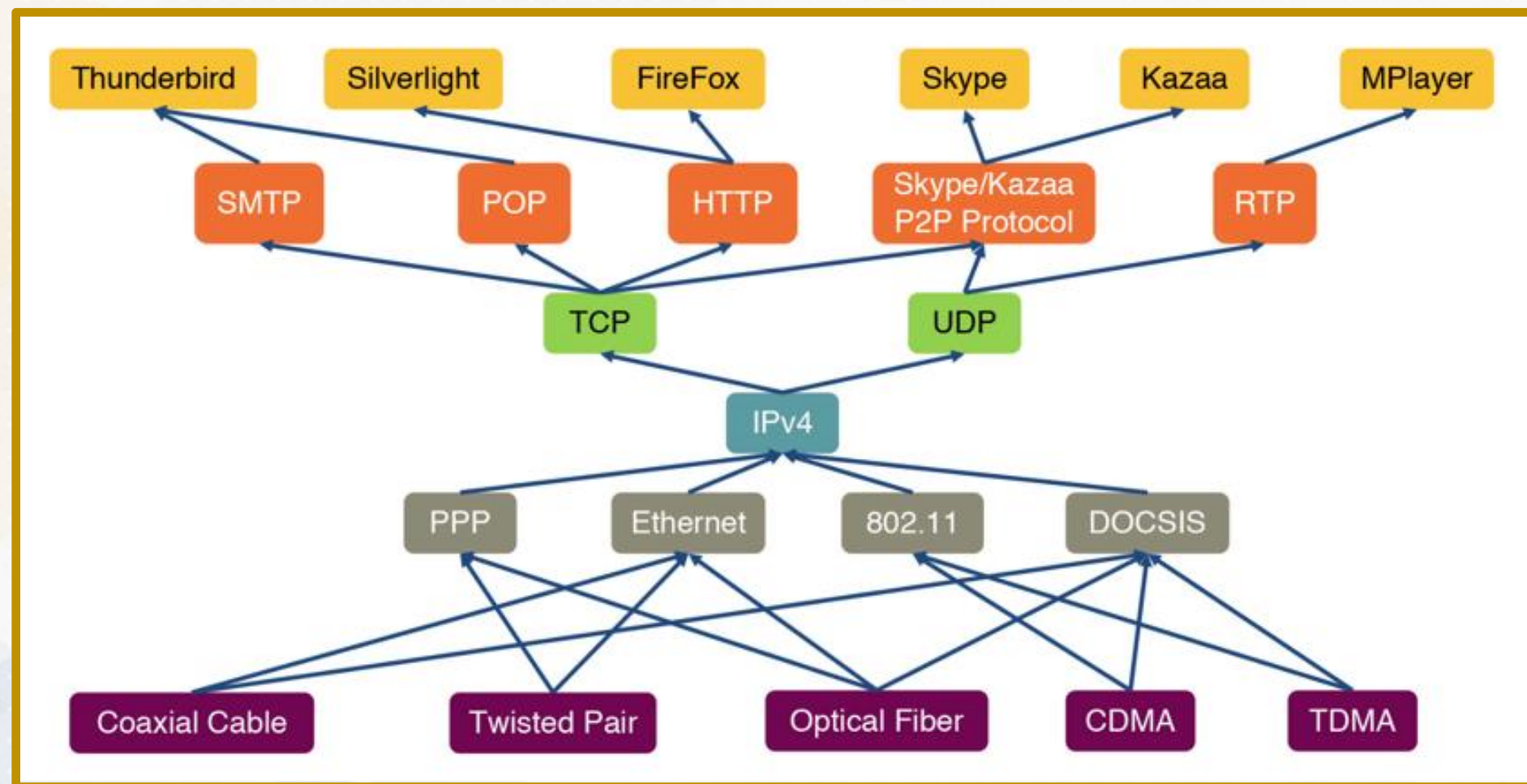
# Network Address Translation

| NAT translation table |                |
|-----------------------|----------------|
| WAN side              | LAN side       |
| 120.70.39.4, 5001     | 10.0.0.4, 3345 |
| ...                   | ...            |





# Why the Internet Protocol Stack has an Hourglass Shape?



Why the **middle of the stack** has been stable while the **edges keep innovating**?

What that means for building **future protocols**?



# Paper

- Why the Internet layered protocol stack evolved into an **“hourglass”** shape, with lots of diversity at the bottom and top but **a narrow, stable “ossified” waist in the middle?**
- Why a few core protocols (e.g., IP/TCP/UDP) can **outlast competitors for so long?**
- What are these findings imply for designing more evolvable **future architectures?**

## The Evolution of Layered Protocol Stacks Leads to an Hourglass-Shaped Architecture\*

Saamer Akhshabi  
College of Computing  
Georgia Institute of Technology  
sakhshab@cc.gatech.edu

Constantine Dovrolis  
College of Computing  
Georgia Institute of Technology  
dovrolis@cc.gatech.edu

### ABSTRACT

The Internet protocol stack has a layered architecture that resembles an hourglass. The lower and higher layers tend to see frequent innovations, while the protocols at the waist of the hourglass appear to be “ossified”. We propose *EvoArch*, an abstract model for studying protocol stacks and their evolution. *EvoArch* is based on a few principles about layered network architectures and their evolution in a competitive environment where protocols acquire value based on their higher layer applications and compete with other protocols at the same layer. *EvoArch* produces an hourglass structure that is similar to the Internet architecture from general initial conditions and in a robust manner. It also suggests a plausible explanation why some protocols, such as TCP or IP, managed to survive much longer than most other protocols at the same layers. Furthermore, it suggests ways to design more competitive new protocols and more evolvable future Internet architectures.

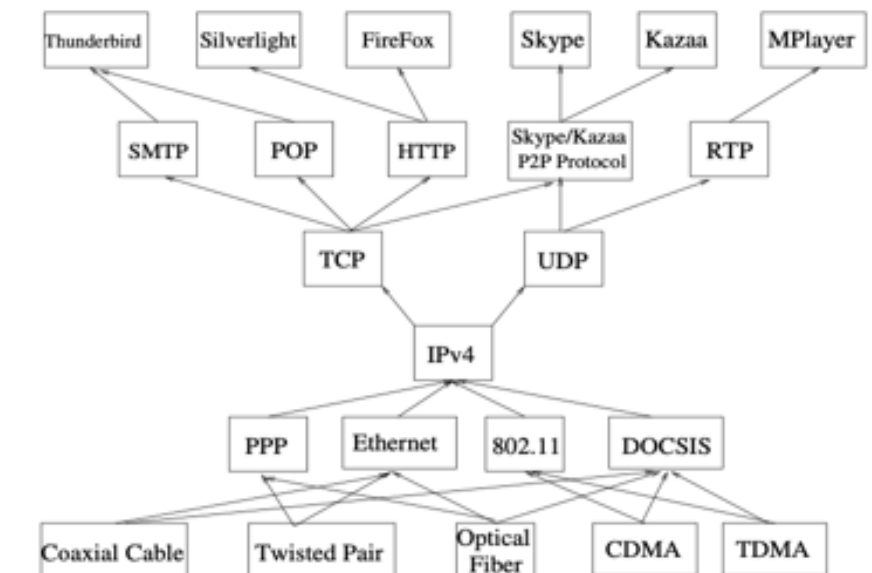
**Categories and Subject Descriptors:** C.2.5 [Computer Communication Networks]: Internet

**General Terms:** Theory

**Keywords:** Internet Architecture, Future Internet, Layering, Network Science, Evolutionary Kernels, Evolution.

### 1. INTRODUCTION

*Why does the Internet protocol stack resemble an hourglass? Is*

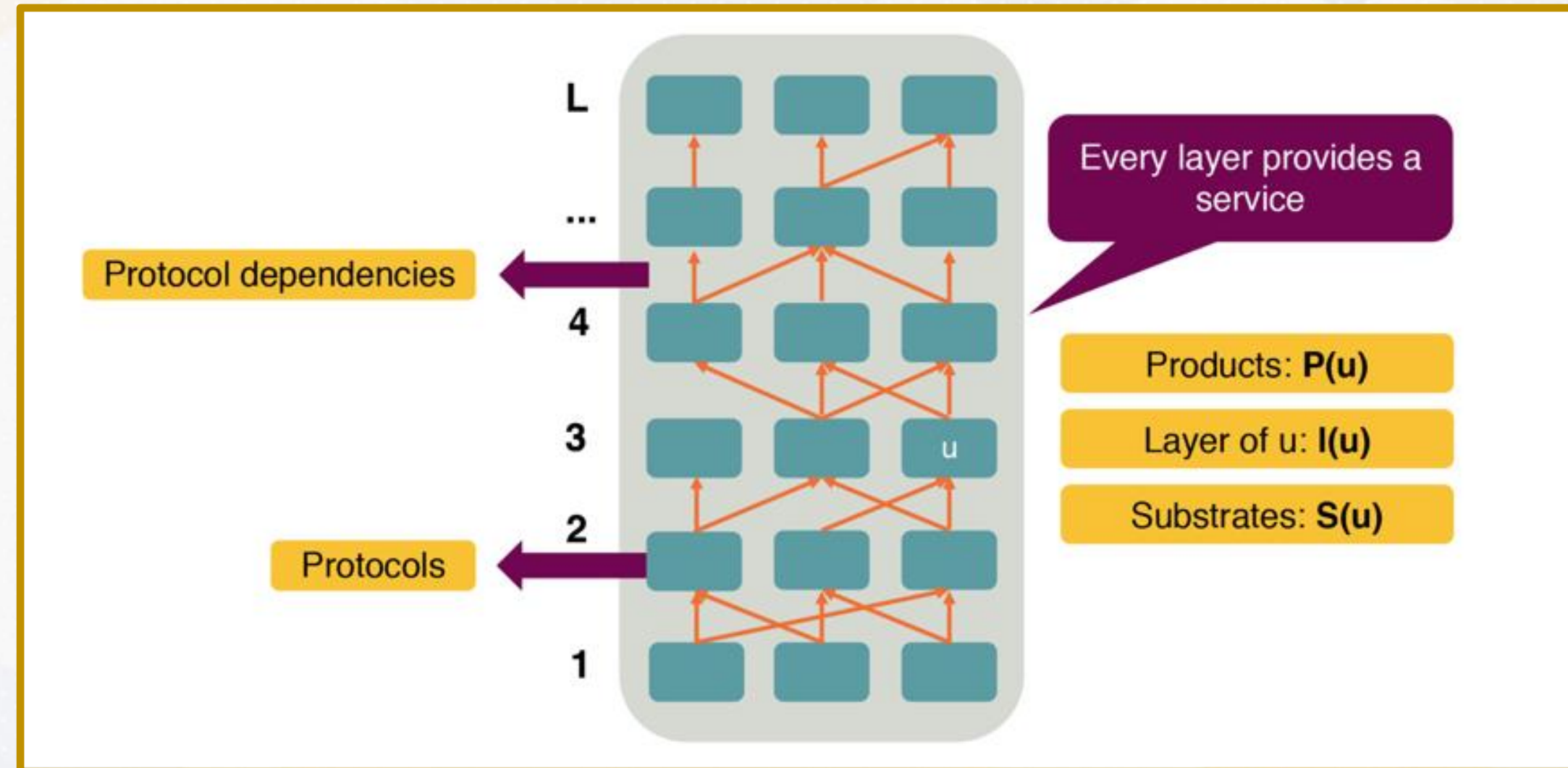
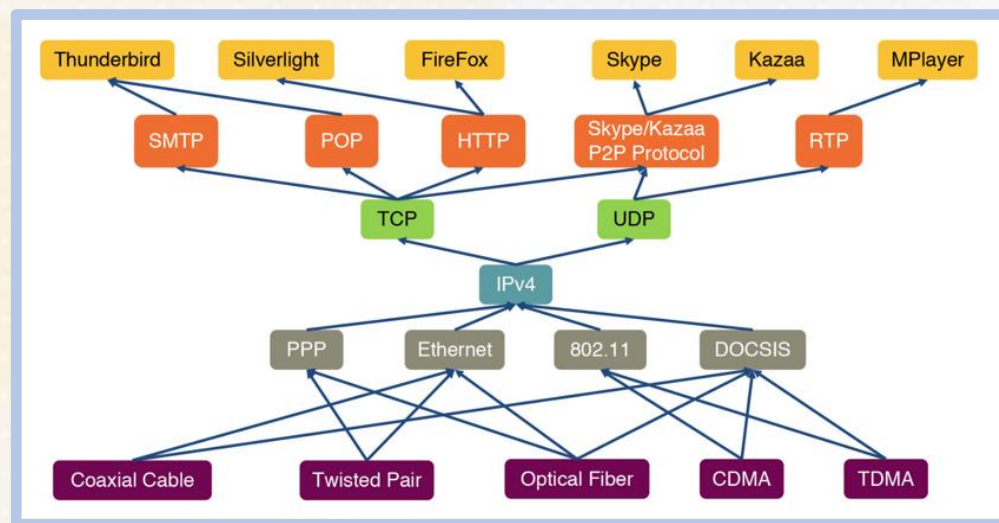


**Figure 1:** An (incomplete) illustration of the hourglass Internet architecture.

Another important question is: *why do we tend to see more frequent innovations at the lower or higher layers of the protocol hourglass, while the protocols at the waist of the hourglass appear to be “ossified” and difficult to replace?* During the last 30–40 years we have seen many new physical and data link layer protocols created and surviving. And of course the same can be said about applications and application-layer protocols. On the other hand,



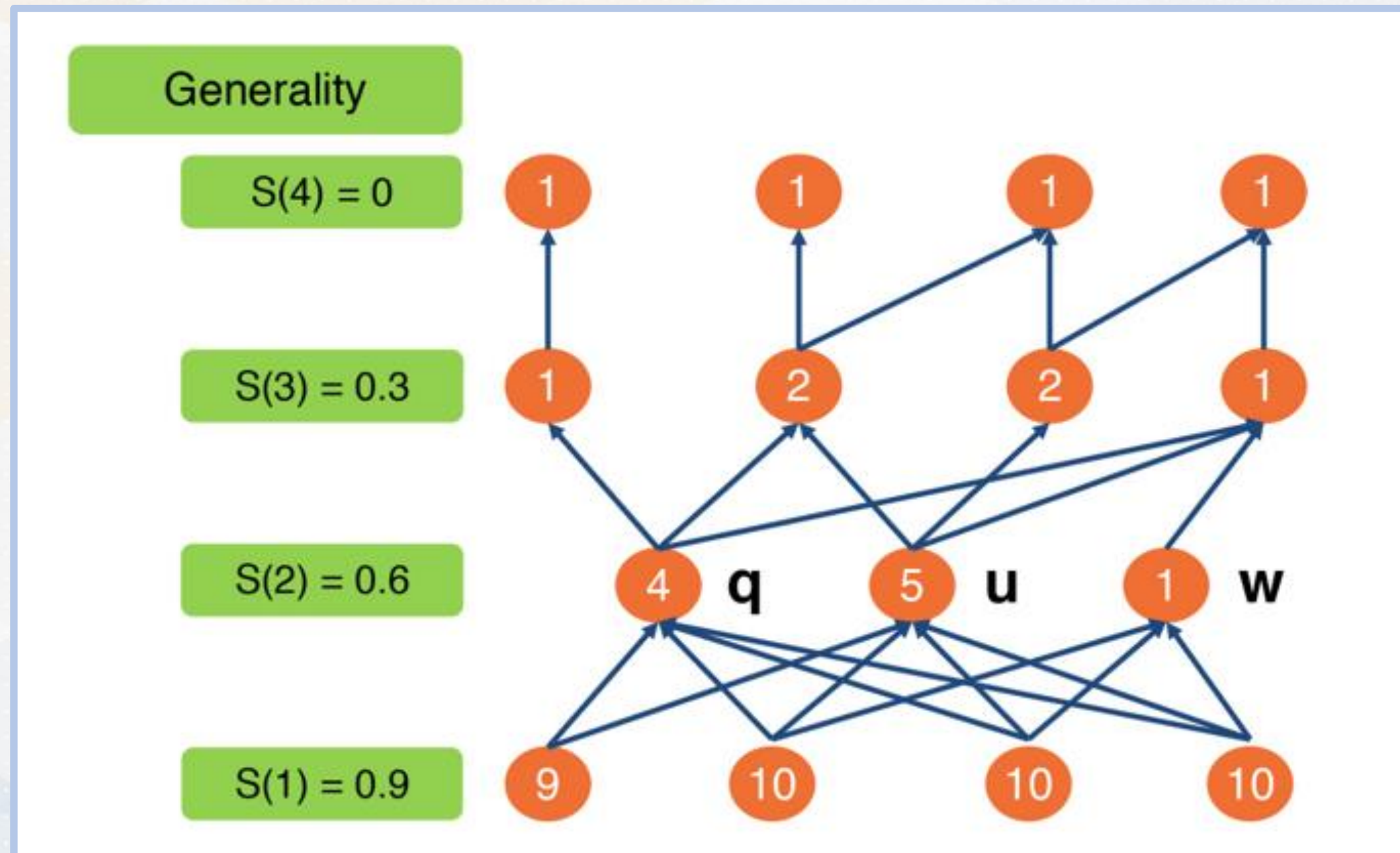
# The Evolution of Layered Protocol Stacks Leads to an Hourglass Architecture (1/4)



Akhshabi, Saamer, and Constantine Dovrolis. "The evolution of layered protocol stacks leads to an hourglass-shaped architecture." *Proceedings of the ACM SIGCOMM 2011 Conference*. 2011.



# The Evolution of Layered Protocol Stacks Leads to an Hourglass Architecture (2/4)



Akhshabi, Saamer, and Constantine Dovrolis. "The evolution of layered protocol stacks leads to an hourglass-shaped architecture." *Proceedings of the ACM SIGCOMM 2011 Conference*. 2011.



# The Evolution of Layered Protocol Stacks Leads to an Hourglass Architecture (3/4)

- The **Hourglass Shape**: The Internet has lots of variety at the bottom (different link technologies) and at the top (many apps), but only a few “core” protocols in the middle.
- EvoArch suggests that **a protocol stays alive when many other protocols and applications rely on it**, and it gets pushed out when it competes directly with a stronger alternative protocol.
- **TCP/IP succeeded early** because it started with a small set of useful data applications, built adoption there, and then more and more applications started depending on it.
- **Once the core has multiple services** and applications built around it, **swapping it out is extremely difficult** because so much would have to change at the same time.



# The Evolution of Layered Protocol Stacks Leads to an Hourglass Architecture (4/5)

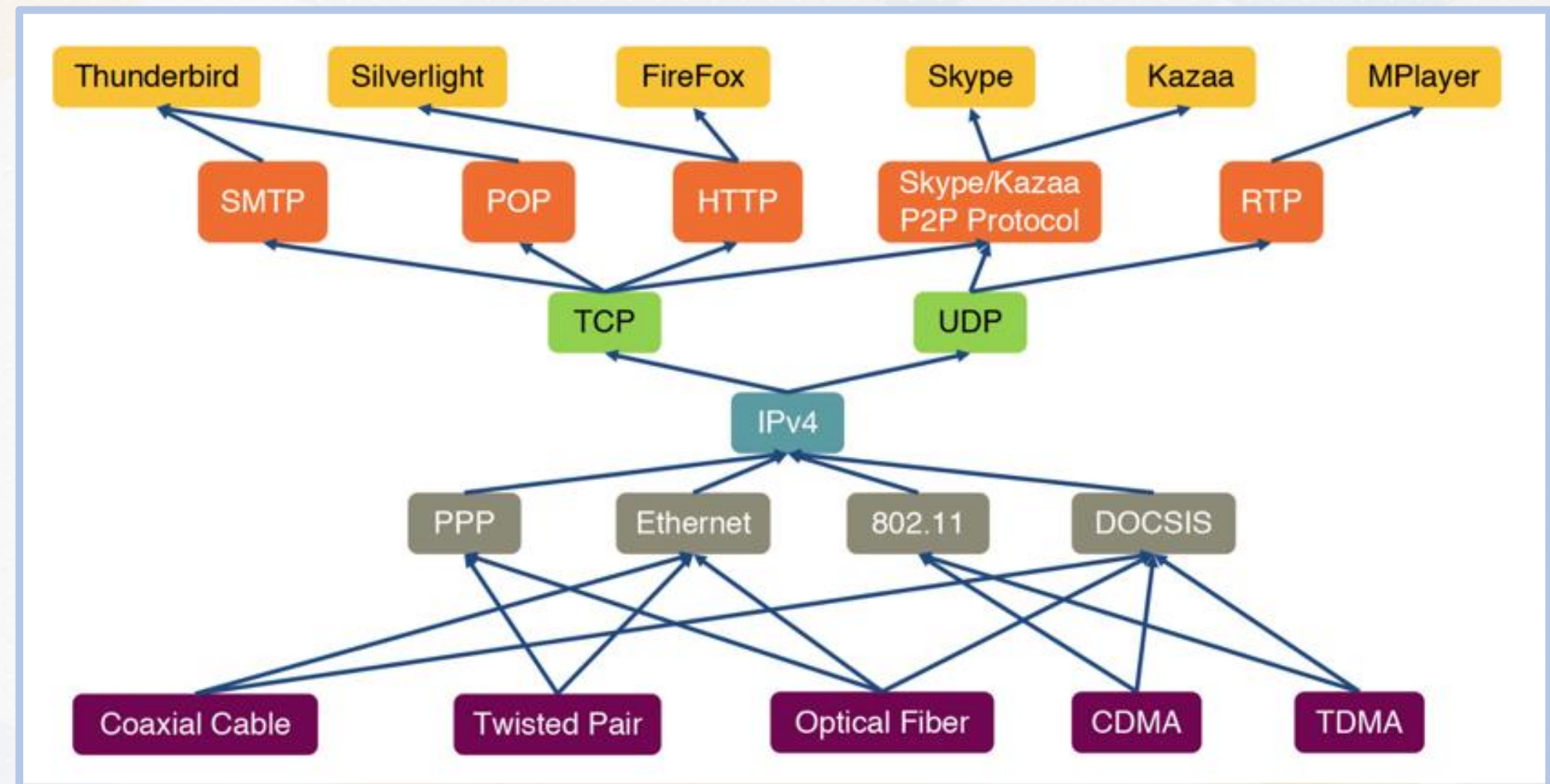
## Looking Ahead

- **Ossification** happens because of how the Internet is designed, not just by chance.
- TCP and UDP are so widely used that they make it **hard for a new network-layer protocol to replace IPv4**.
- New “better” protocols don’t win unless lots of real applications and systems **actually adopt them**.
- **EvoArch argues that even new architectures tend to evolve into an hourglass** again; therefore we need to design for multiple core options and smooth ways for new protocols to coexist.



# Interconnecting Hosts (1/2)

Networks use **different interconnection devices** that provide different services and operate at different layers.



Akhshabi, Saamer, and Constantine Dovrolis. "The evolution of layered protocol stacks leads to an hourglass-shaped architecture." *Proceedings of the ACM SIGCOMM 2011 Conference*. 2011.



# Interconnecting Hosts (2/2)

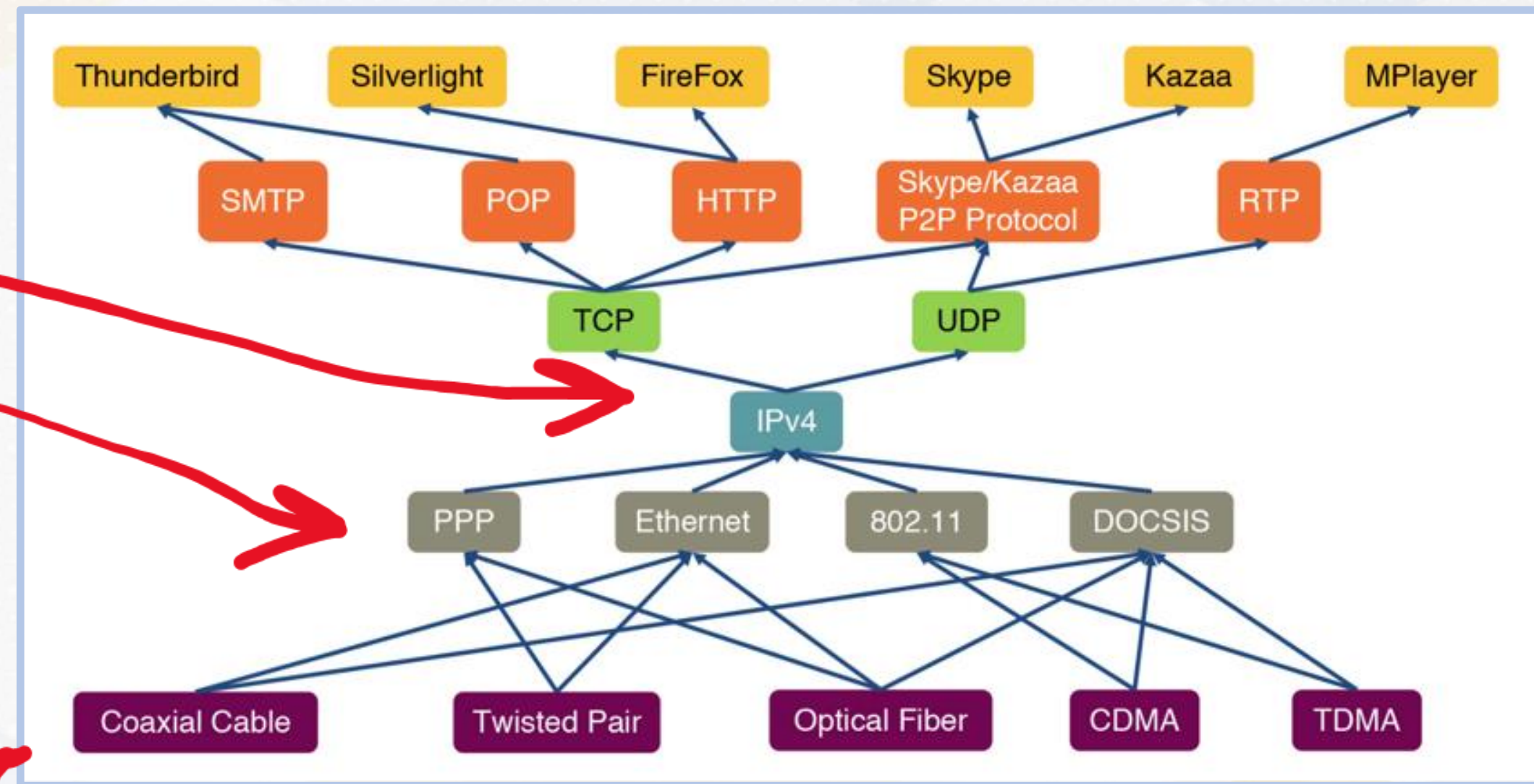
## Routers and Layer-3

**switches:** forward packets across networks at the network layer; routing details come later.

## Bridges and Layer-2

**switches:** forward frames based on MAC addresses to connect hosts not directly connected on the same LAN.

**Repeaters and hubs (Layer-1):** receive and forward bits/signals to extend Ethernet segments; simple, inexpensive, can be arranged hierarchically.



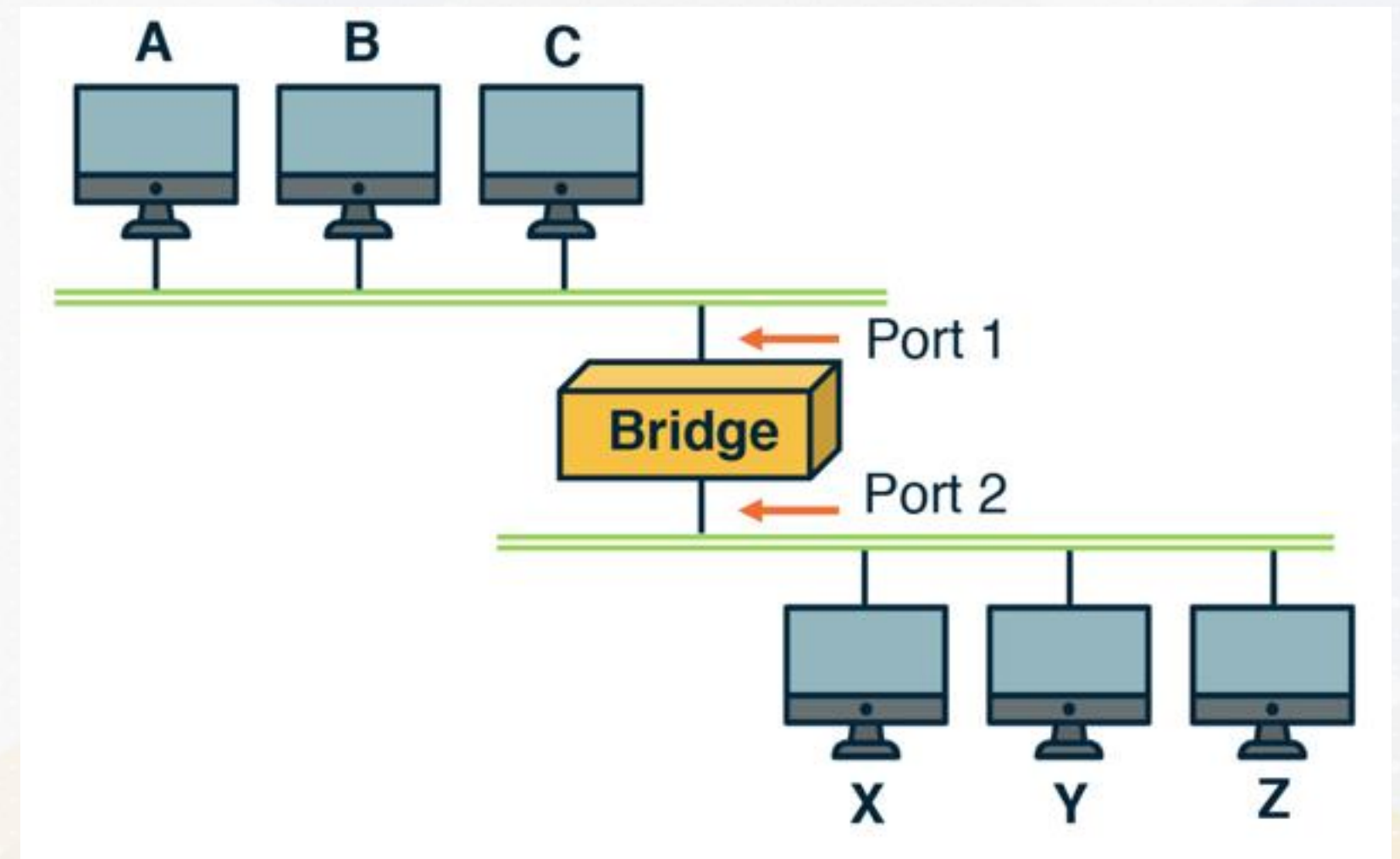
Akhshabi, Saamer, and Constantine Dovrolis. "The evolution of layered protocol stacks leads to an hourglass-shaped architecture." *Proceedings of the ACM SIGCOMM 2011 Conference*. 2011.



# Learning Bridges (1/2)

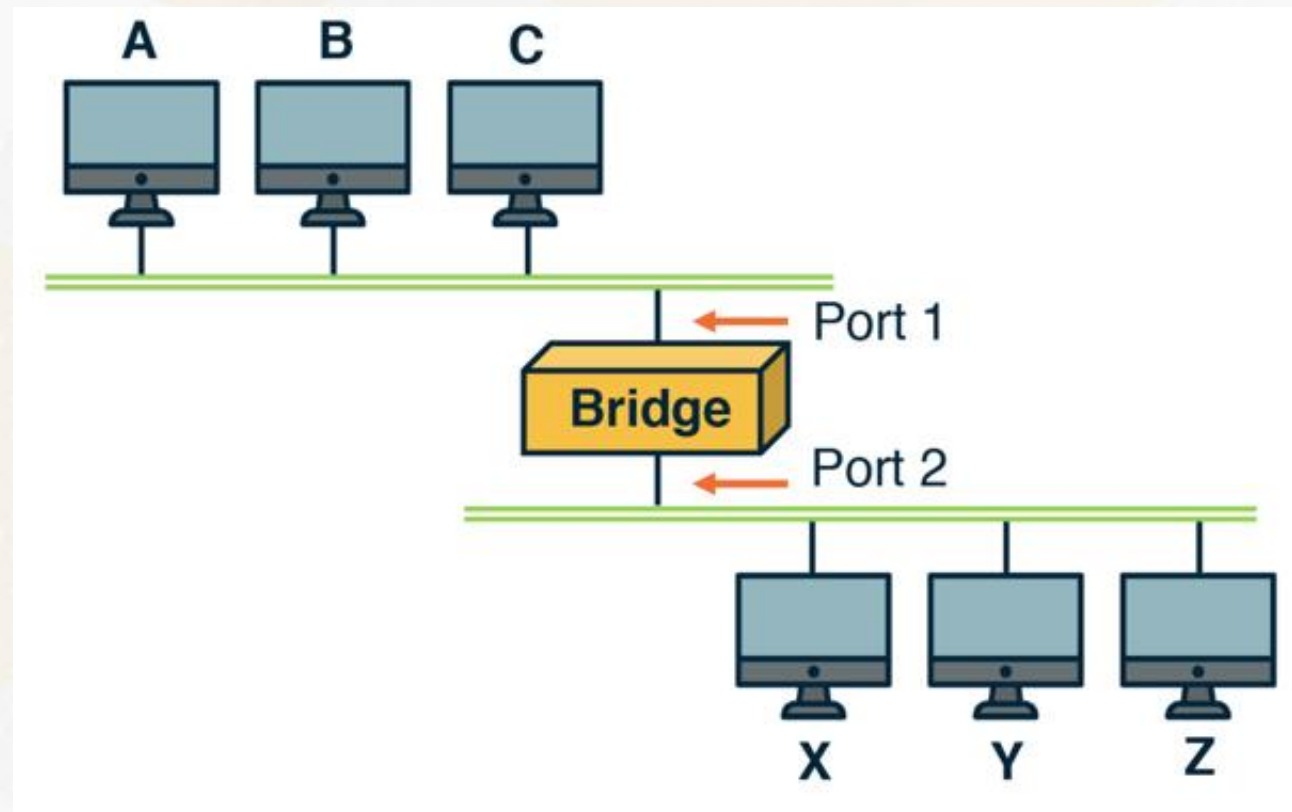
A bridge has multiple ports and forwards frames from an input to one or more outputs, but not necessarily to all ports.

**Learning goal:** Build and maintain a forwarding table so frames are sent only on the correct port(s). Every received frame is a learning opportunity; the bridge records “source MAC → incoming port.”





# Learning Bridges (2/2)



| Host | Port |
|------|------|
| A    | 1    |
| B    | 1    |
| C    | 1    |
| X    | 2    |
| Y    | 2    |
| Z    | 2    |

**Forwarding behavior:** If destination MAC is known, forward only to the mapped port; if unknown, flood to all except the incoming port.

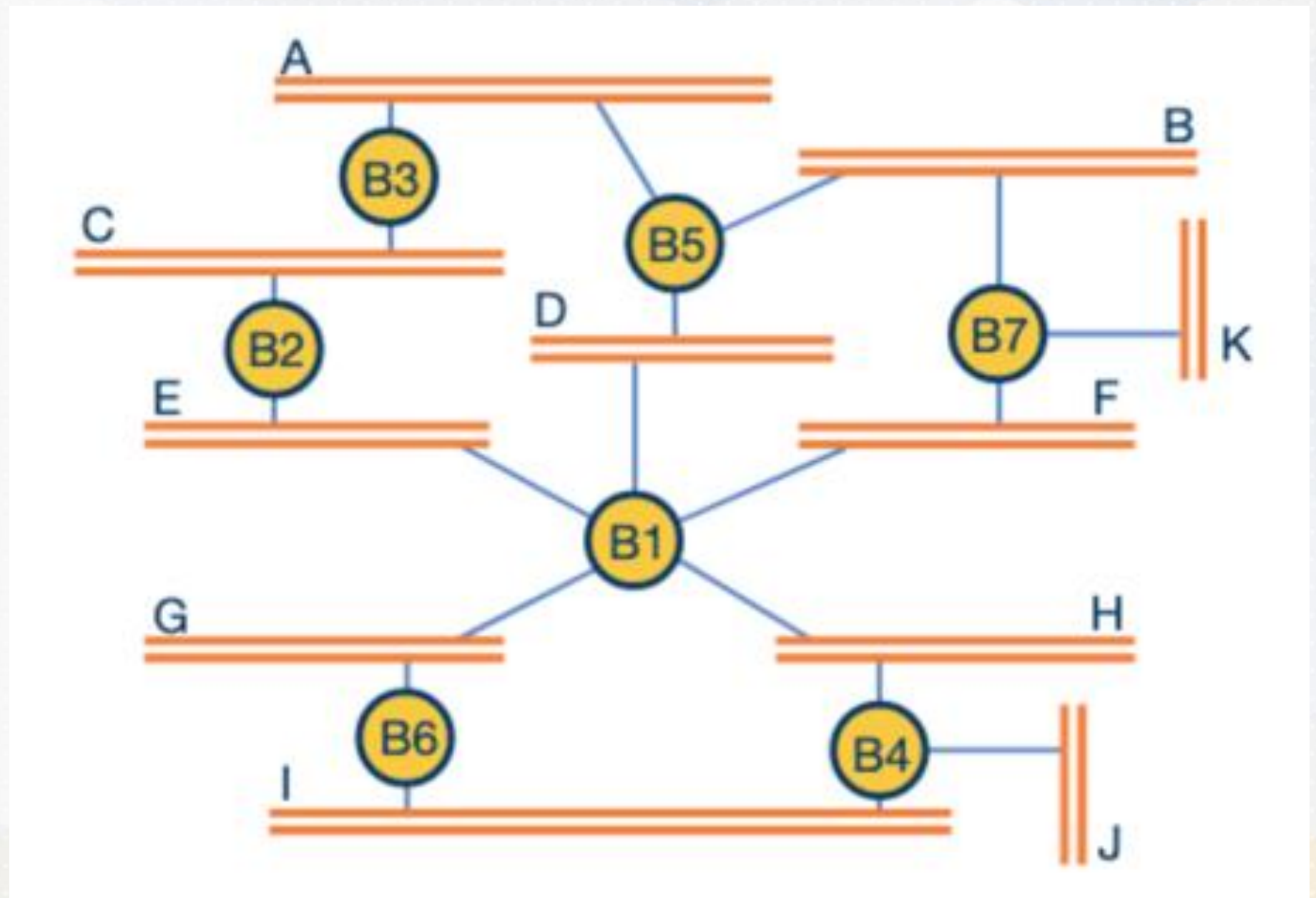
**Practical limitation:** Output ports have finite bandwidth; when arrival rate exceeds output capacity, frames queue in buffers and may be dropped if buffers fill.



# The Spanning Tree Algorithm (1/2)

Bridged LAN topologies with cycles can create forwarding loops, causing frames to circulate indefinitely.

Spanning tree fixes this by disabling selected links/ports so the active topology is loop-free.

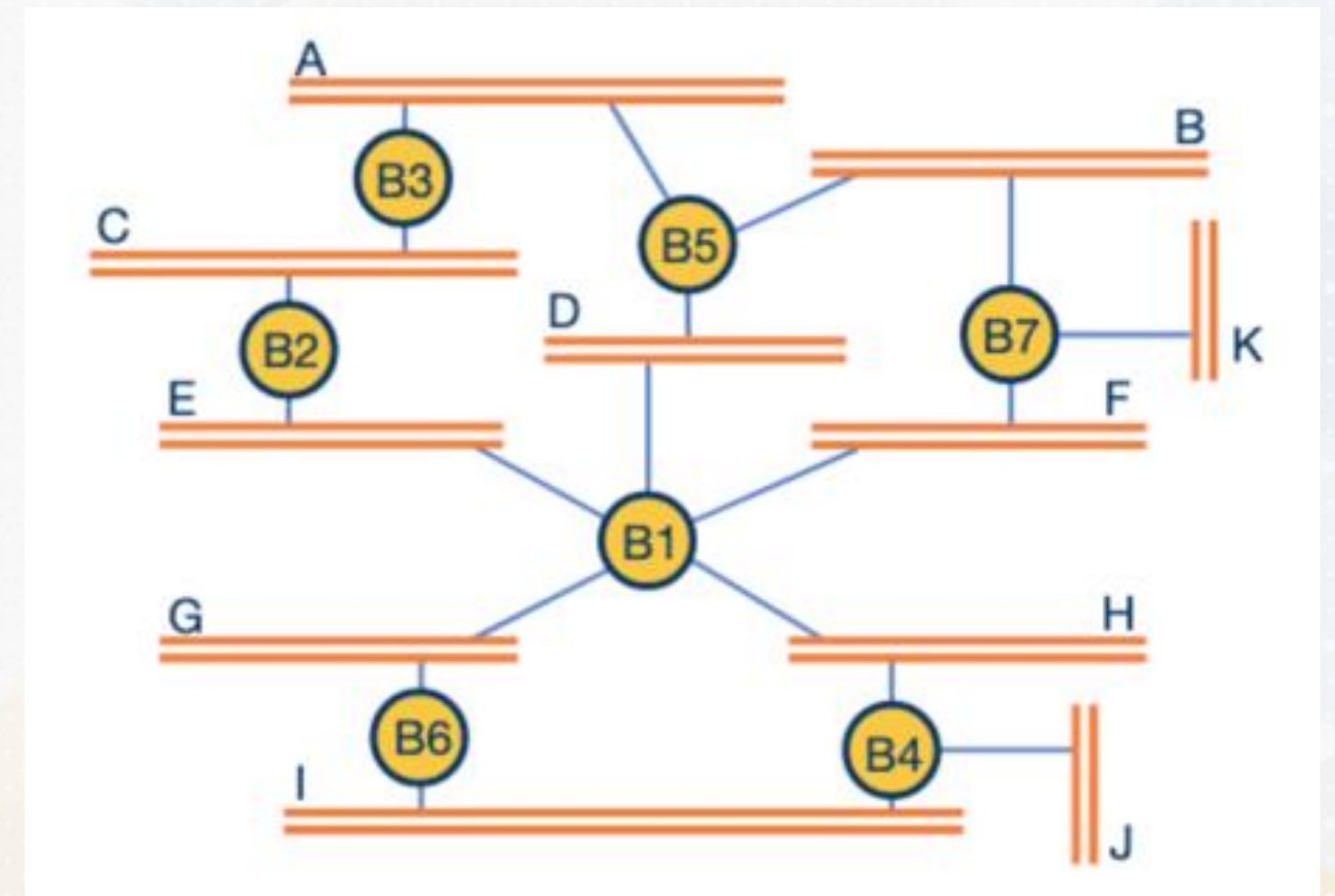


# The Spanning Tree Algorithm (2/2)

Model the network as a graph: bridges are nodes, links between bridges are edges; the algorithm builds a spanning tree rooted at one bridge.

**Distributed process:** in rounds, bridges exchange configuration messages with (sender ID, perceived root ID, distance to root).

**Selection rules:** prefer lower root ID; if tied, prefer smaller distance; if still tied, prefer lower sender ID; ports that lead “away” from the best path stop forwarding to prevent loops.





# Summary

- **Architectural design choices** – such as the layered protocol stack and the **end-end-principle** – make the Internet modular and evolvable.
- **The Internet's hourglass architecture** explains why a few core protocols dominate. Researchers suggest an evolutionary model that explains how the ecosystem dependence makes it hard to replace.
- These design ideas show up in practice. For example **devices operating on different layers (such as switches, routers)**, and algorithms such as the spanning tree protocol, enable connectivity (while preventing loops).